

Credit Card Approval Prediction System Using Machine Learning

Project Description:

The Credit Card Approval Prediction System is a Machine Learning-based web application developed to predict whether a customer's credit card application is likely to be approved or rejected based on personal, financial, and employment-related information. The application assists banks and financial institutions in making quick and accurate credit approval decisions while reducing manual effort.

The system is developed using Python, Flask, Scikit-learn, Pandas, NumPy, SQLite, HTML, and CSS. It utilizes a trained Random Forest Classifier to analyze applicant details such as annual income, education level, employment status, family status, age, occupation, housing type, and other financial attributes.

The application includes a secure user authentication system where authorized users can log in before accessing the prediction system. After successful authentication, users can enter applicant details through an easy-to-use web interface. The machine learning model processes the information and instantly predicts whether the applicant is a Good Customer (Approved) or Bad Customer (Rejected) along with a confidence score.

Every prediction is automatically stored in an SQLite database, allowing users to view previous applicant records through the Prediction History page. The application also provides a Dashboard that displays prediction analytics such as total applications processed, approved customers, rejected customers, approval rate, average prediction confidence, and overall model statistics.

The project demonstrates the integration of Machine Learning, Database Management, and Web Development to build an intelligent decision-support system for credit card approval prediction with improved usability, security, and data management.

Scenarios

Scenario 1: Credit Card Approval Prediction

A bank employee logs into the system and enters an applicant's personal, financial, and employment details. The trained Random Forest model analyzes the information and predicts whether the applicant is eligible for credit card approval, along with the prediction confidence score.

Scenario 2: Prediction History Management

After every prediction, the applicant's details, prediction result, confidence score, date, and time are automatically stored in the SQLite database. Users can access the **Prediction History** page to review previous prediction records whenever required.

Scenario 3: Dashboard Analytics

The Dashboard retrieves prediction data from the database and displays important analytics such as:

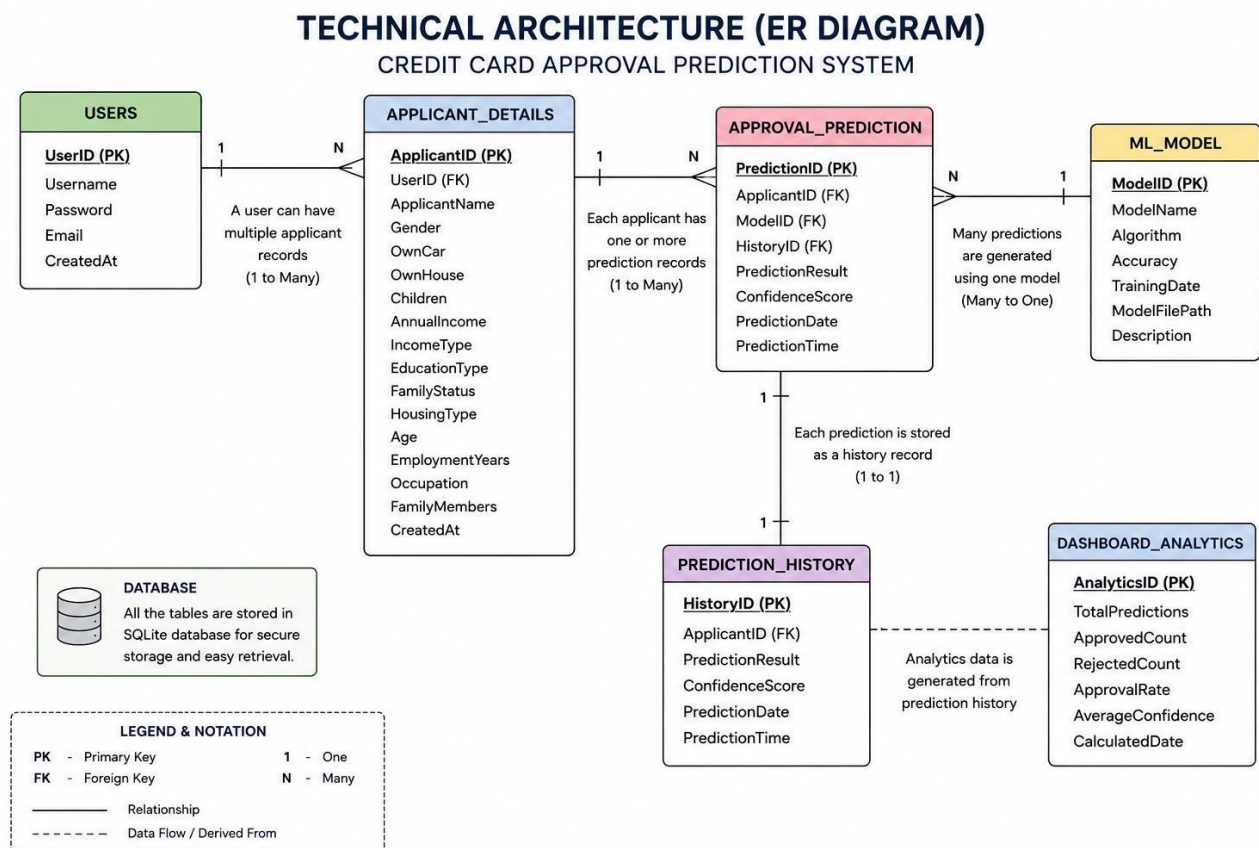
- Total Predictions
- Approved Customers
- Rejected Customers
- Approval Rate
- Average Prediction Confidence
- Total Records Processed

This enables users to monitor the overall performance of the prediction system.

Scenario 4: Secure User Authentication

Only registered users with valid login credentials can access the Credit Card Approval Prediction System. The authentication module verifies user credentials before allowing access to prediction, history, and dashboard functionalities.

Technical Architecture:



Description:

The **Technical Architecture** of the **Credit Card Approval Prediction System** illustrates the interaction between the major components involved in user authentication, applicant data management, machine learning prediction, database storage, and analytics.

The **Users** entity stores the login credentials of authorized users who can access the application. After successful authentication, users enter the applicant's personal, financial, and employment details through the web interface. These details are stored in the **Applicant_Details** entity, which acts as the primary source of information for the prediction process.

The **ML_Model** entity represents the trained **Random Forest Classifier** used to predict whether a credit card application should be approved or rejected. The model processes the applicant's information and generates a prediction along with a confidence score.

The generated prediction is stored in the **Approval_Prediction** entity, which contains the prediction result, confidence score, prediction date, and prediction time. Each prediction is then recorded in the **Prediction_History** entity, enabling users to review all previously generated predictions.

The **Dashboard_Analytics** entity retrieves data from the Prediction History to generate useful analytical information such as the total number of predictions, approved customers, rejected customers, approval rate, and average prediction confidence. These analytics help users monitor the overall performance of the system.

All application data, including user credentials, applicant details, prediction records, and analytics, are securely stored in the **SQLite Database**, ensuring reliable data management and easy retrieval. The integration of **Flask**, **SQLite**, and the **Random Forest Machine Learning model** enables the application to provide accurate predictions, maintain historical records, and present meaningful analytical insights through a user-friendly web interface.

Pre-requisites

- Python 3.10 or later installed
- Visual Studio Code (VS Code)
- Flask Framework
- NumPy Library
- Pandas Library
- Scikit-learn Library
- Matplotlib Library
- Seaborn Library
- SQLite3 Database (included with Python)
- HTML5 and CSS3
- Git (optional, for version control)
- Google Chrome or Microsoft Edge web browser
- Stable internet connection (for installing Python packages)

Project Workflow:

Milestone 1: Dataset Preparation and Requirement Analysis

- Activity 1.1: Collect the credit card approval dataset from open-source repositories and understand its features and target variable.
- Activity 1.2: Perform data preprocessing by handling missing values, removing duplicates, encoding categorical variables, and preparing the dataset for training.
- Activity 1.3: Perform exploratory data analysis (EDA) using statistical summaries and visualization techniques to understand data distribution.
- Activity 1.4: Design the overall system architecture, including the Flask application, SQLite database, machine learning model, and dashboard.

Milestone 2: Machine Learning Model Development

- Activity 2.1: Split the dataset into training and testing sets.
- Activity 2.2: Train multiple machine learning algorithms, including Logistic Regression, Decision Tree, Random Forest, and XGBoost.
- Activity 2.3: Evaluate the models using Accuracy, Precision, Recall, and F1-Score.
- Activity 2.4: Select the best-performing model (Random Forest) and save it as model.pkl for deployment.

Milestone 3: Backend Development

- Activity 3.1: Develop the Flask backend by creating application routes and integrating the trained machine learning model.
- Activity 3.2: Implement user authentication (Login) for secure system access.
- Activity 3.3: Integrate the SQLite database to store prediction records and user information.
- Activity 3.4: Implement prediction logic, confidence score calculation, and result generation.

Milestone 4: Frontend Development

- Activity 4.1: Design a responsive web interface using HTML and CSS for applicant data entry.
- Activity 4.2: Develop the prediction result page displaying approval status and confidence score.
- Activity 4.3: Develop the Prediction History page to display previously stored prediction records.
- Activity 4.4: Develop the Dashboard page to display analytics such as total predictions, approved customers, rejected customers, approval rate, and average confidence.

Milestone 5: Testing and Deployment

- Activity 5.1: Perform functional testing of the login system, prediction module, prediction history, and dashboard.
 - Activity 5.2: Validate the prediction accuracy and verify database operations.
 - Activity 5.3: Execute the application on the local Flask server and ensure smooth end-to-end functionality.
-

Milestone 6: Documentation and Future Enhancements

- Activity 6.1: Prepare project documentation, reports, and demonstration materials.
- Activity 6.2: Summarize the project outcomes and evaluate overall system performance.
- Activity 6.3: Identify future enhancements such as cloud deployment, advanced dashboard visualizations, stronger authentication mechanisms, and database upgrades to MySQL or PostgreSQL.

Milestone 1: Dataset Preparation and System Architecture

In this milestone, the primary objective is to prepare the dataset, perform data preprocessing, design the overall system architecture, and configure the development environment for the Credit Card Approval Prediction System. The credit card applicant dataset is collected and analyzed to understand the available features and target variable. Data preprocessing techniques are applied to improve data quality before training the machine learning model.

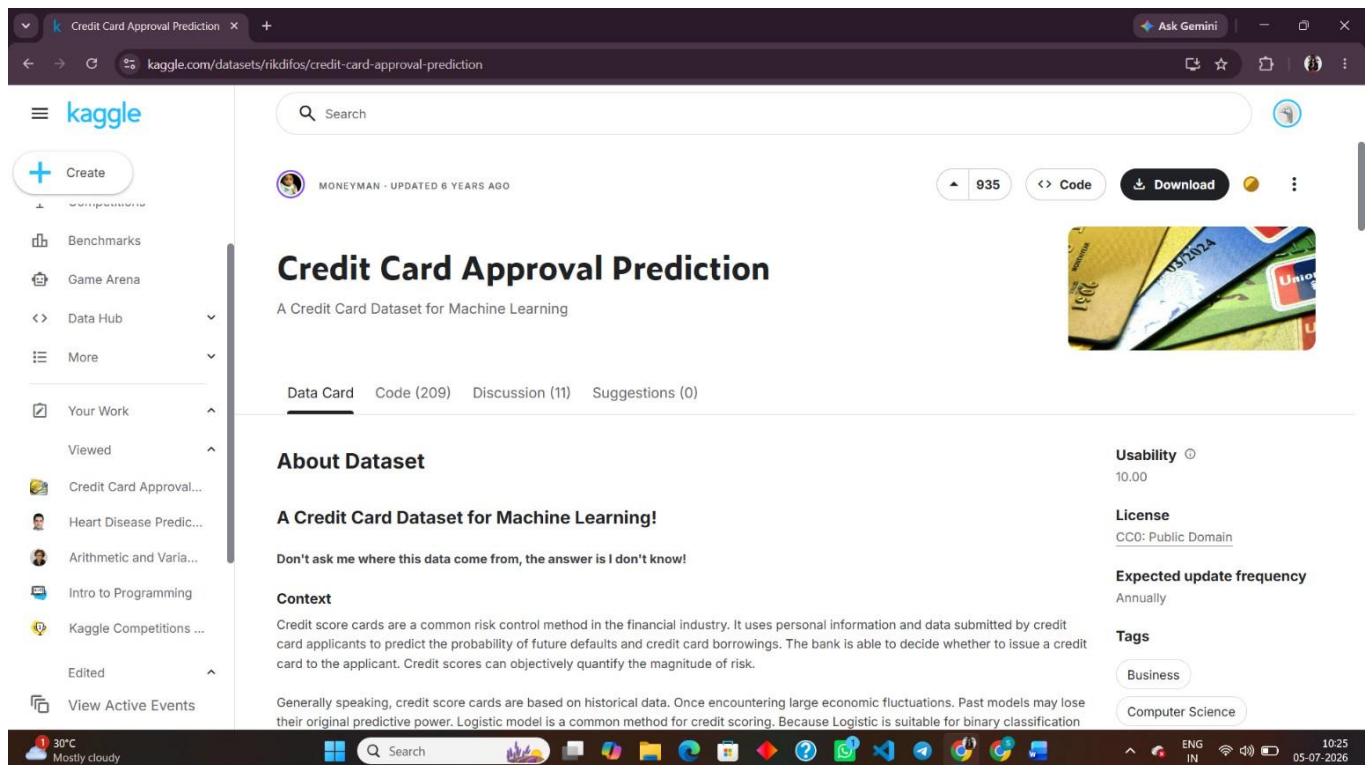
The overall system architecture is designed to illustrate the interaction between the Flask web application, SQLite database, trained Random Forest model, prediction history, and dashboard analytics. Finally, the development environment is configured by installing the required Python libraries and project dependencies.

Activity 1.1: Collect and Prepare the Dataset

Before building the machine learning model, the applicant and credit history datasets must be collected and prepared.

Step 1 — Download the Dataset

Download the **Application Record** and **Credit Record** datasets from the Kaggle Credit Card Approval Prediction Dataset.



The screenshot shows the Kaggle dataset page for "Credit Card Approval Prediction". The page is titled "Credit Card Approval Prediction" and is described as "A Credit Card Dataset for Machine Learning". It is updated 6 years ago by "MONEYMAN". The page has 935 views, 209 code files, 11 discussions, and 0 suggestions. The "About Dataset" section states: "A Credit Card Dataset for Machine Learning! Don't ask me where this data come from, the answer is I don't know!". The "Context" section explains that credit score cards are a common risk control method in the financial industry, used to predict the probability of future defaults and credit card borrowings. The "Usability" is 10.00, the "License" is CC0: Public Domain, and the "Expected update frequency" is Annually. The "Tags" are Business and Computer Science.

Step 2 — Load the Dataset

Load both datasets into Python using the Pandas library.

```
import pandas as pd
application = pd.read_csv("application_record.csv")
credit = pd.read_csv("credit_record.csv")
```

```
# =====
# DATA COLLECTION
# =====

print("=" * 60)
print("LOADING DATASETS")
print("=" * 60)

application = pd.read_csv("application_record.csv")
credit = pd.read_csv("credit_record.csv")

print("\nApplication Dataset")
print(application.head())

print("\nCredit Dataset")
print(credit.head())

print("\nApplication Shape :", application.shape)
print("Credit Shape      :", credit.shape)

# =====
```

Step 3 — Explore the Dataset

Display the first few records, check missing values, and understand the available features.

```
application.head()
credit.head()
application.info()
```

```
• (.venv) PS C:\Users\johns\Downloads\Credit_Card_Project\5. Project Development Phase\Source Code> python main.py
=====
LOADING DATASETS
=====

Application Dataset
  ID  CODE  GENDER  FLAG_OWN_CAR  FLAG_OWN_REALTY  CNT_CHILDREN  ...  FLAG_WORK_PHONE  FLAG_PHONE  FLAG_EMAIL  OCCUPATION_TYPE  CNT_FAM_MEMBERS
0  5008804      M      Y      Y      Y      0  ...      1      0      0      NaN      2.0
1  5008805      M      Y      Y      Y      0  ...      1      0      0      NaN      2.0
2  5008806      M      Y      Y      Y      0  ...      0      0      0  Security staff      2.0
3  5008808      F      N      Y      Y      0  ...      0      1      1  Sales staff      1.0
4  5008809      F      N      Y      Y      0  ...      0      1      1  Sales staff      1.0

[5 rows x 18 columns]

Credit Dataset
  ID  MONTHS_BALANCE  STATUS
0  5001711           0      X
1  5001711          -1      0
2  5001711          -2      0
3  5001711          -3      0
4  5001712           0      C

Application Shape : (438557, 18)
Credit Shape      : (1048575, 3)
```


Step 4 — Perform Data Preprocessing

Clean the dataset by:

- Handling missing values
- Removing duplicate records
- Encoding categorical variables
- Selecting useful features
- Preparing the final dataset for model training

```
(.venv) PS C:\Users\johns\Downloads\Credit_Card_Project\5. Project Development Phase\Source Code> python main.py
=====
DATA PREPROCESSING
=====

Missing Values After Cleaning
ID                0
CODE_GENDER       0
FLAG_OWN_CAR      0
FLAG_OWN_REALTY   0
CNT_CHILDREN      0
AMT_INCOME_TOTAL  0
NAME_INCOME_TYPE  0
NAME_EDUCATION_TYPE 0
NAME_FAMILY_STATUS 0
NAME_HOUSING_TYPE 0
DAYS_BIRTH        0
DAYS_EMPLOYED     0
FLAG_MOBIL        0
FLAG_WORK_PHONE   0
FLAG_PHONE        0
FLAG_EMAIL        0
OCCUPATION_TYPE   0
CNT_FAM_MEMBERS   0
dtype: int64

Credit Dataset After Grouping
   ID  TARGET
0  5001711    0
1  5001712    0
2  5001713    0
3  5001714    0
4  5001715    0

Merged Dataset Shape : (36457, 19)
   ID  CODE_GENDER  FLAG_OWN_CAR  FLAG_OWN_REALTY  CNT_CHILDREN  ...  FLAG_PHONE  FLAG_EMAIL  OCCUPATION_TYPE  CNT_FAM_MEMBERS  TARGET
0  5008804         M           Y           Y           0  ...           0           0         Unknown           2.0         1
1  5008805         M           Y           Y           0  ...           0           0         Unknown           2.0         1
2  5008806         M           Y           Y           0  ...           0           0  Security staff           2.0         0
3  5008808         F           N           Y           0  ...           1           1    Sales staff           1.0         0
4  5008809         F           N           Y           0  ...           1           1    Sales staff           1.0         0
```

Activity 1.2: Design the System Architecture

The architecture of the Credit Card Approval Prediction System is designed to integrate machine learning, database management, and web technologies.

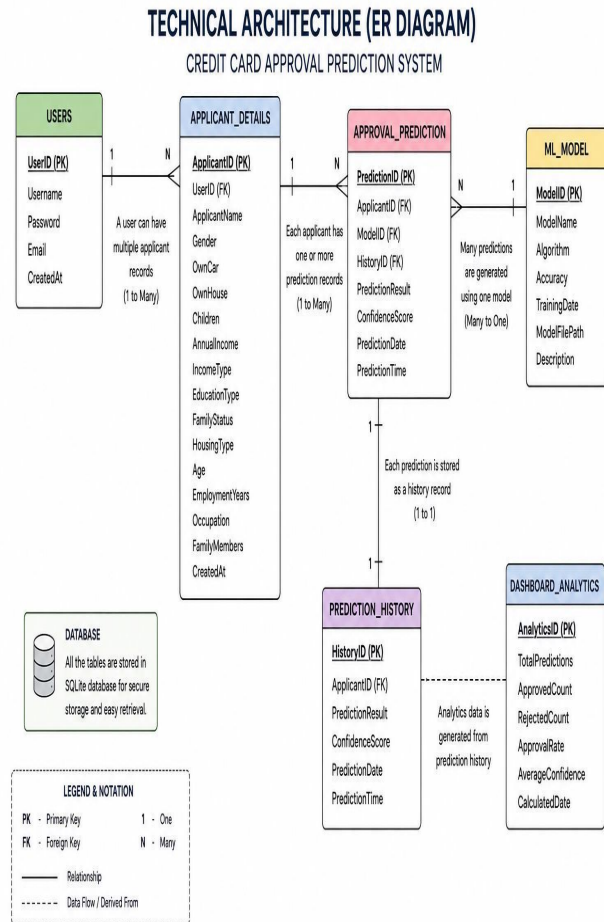
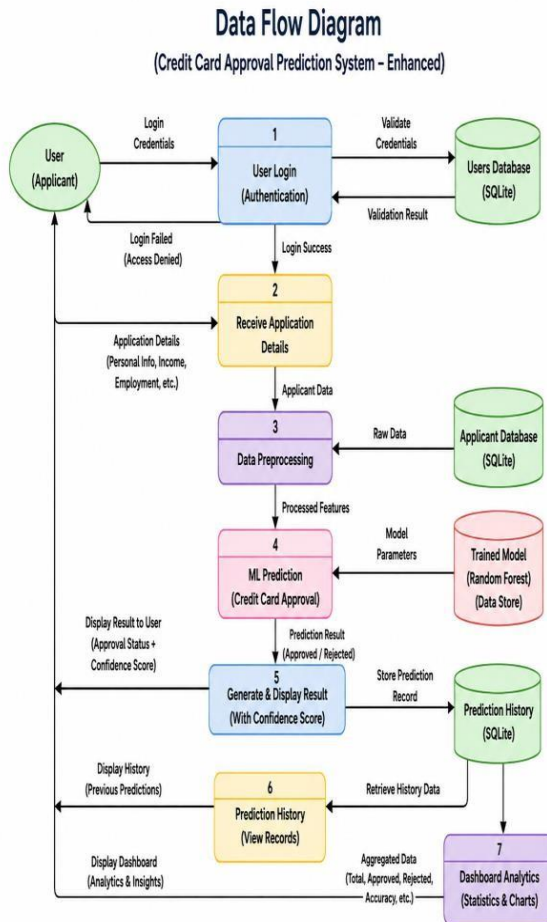
The architecture consists of:

- Flask Web Application
- Login Authentication Module
- Applicant Data Entry Form
- Random Forest Machine Learning Model
- SQLite Database
- Prediction History Module
- Dashboard Analytics Module

The prediction workflow is as follows:

- User logs into the system.

- Applicant details are entered.
- Data is preprocessed.
- Random Forest model predicts the result.
- Prediction and confidence score are displayed.
- Prediction is stored in SQLite.
- Dashboard updates automatically.



Activity 1.3: Configure the Machine Learning Workflow

The machine learning workflow consists of the following stages:

- Import required Python libraries.
- Load the processed dataset.
- Split the dataset into training and testing sets.
- Train multiple machine learning algorithms.
- Compare model performance.
- Select Random Forest as the final model.
- Save the trained model as **model.pkl**.
- Load the model into the Flask application for prediction.


```
=====
RANDOM FOREST
=====

Accuracy : 0.8893
F1 Score : 0.4190

Confusion Matrix
[[6194 240]
 [ 567 291]]

Classification Report
              precision    recall  f1-score   support

     0       0.92       0.96       0.94       6434
     1       0.55       0.34       0.42        858

 accuracy          0.89       0.89       0.89       7292
 macro avg         0.73       0.65       0.68       7292
weighted avg         0.87       0.89       0.88       7292

Generating Occupation Distribution...
Occupation Distribution Saved
=====
```

```
=====
MODEL COMPARISON
=====

              Model  Accuracy  F1 Score
0  Logistic Regression  0.882337  0.000000
1      Decision Tree   0.878086  0.376140
2      Random Forest   0.889331  0.419006
Occupation Distribution Saved
Generating Occupation Distribution...
Occupation Distribution Saved

Top Important Features
              Feature  Importance
9      DAYS_BIRTH     0.259743
10     DAYS_EMPLOYED  0.185590
4      AMT_INCOME_TOTAL 0.161564
15     OCCUPATION_TYPE 0.081524
7      NAME_FAMILY_STATUS 0.041735
16     CNT_FAM_MEMBERS  0.034606
5      NAME_INCOME_TYPE 0.034122
6      NAME_EDUCATION_TYPE 0.033170
13     FLAG_PHONE       0.025263
1      FLAG_OWN_CAR      0.023837
Generating Occupation Distribution...
Occupation Distribution Saved

Best Model Saved Successfully as model.pkl
=====
```

Activity 1.4: Set Up the Development Environment

Install the required software and Python libraries before running the application.

Install Python

Install **Python 3.10 or later** from the official website.

```
(.venv) PS C:\Users\johns\Downloads\Credit_Card_Project\5. Project Development Phase\Source Code> python --version
Python 3.12.10
(.venv) PS C:\Users\johns\Downloads\Credit_Card_Project\5. Project Development Phase\Source Code> |
```

Create a Virtual Environment

python -m venv .venv

```
Command executed now and took 5.179s
PS C:\Users\johns\Downloads\Credit_Card_Project> python -m venv .venv
PS C:\Users\johns\Downloads\Credit_Card_Project> |
```

Activate the Virtual Environment

.venv\Scripts\activate

- PS C:\Users\johns\Downloads\Credit_Card_Project> python -m venv .venv
- PS C:\Users\johns\Downloads\Credit_Card_Project> .\.venv\Scripts\activate
- (.venv) PS C:\Users\johns\Downloads\Credit_Card_Project> █

Install Required Libraries

pip install -r requirements.txt

or

pip install flask numpy pandas scikit-learn matplotlib seaborn

```
(.venv) PS C:\Users\johns\Downloads\Credit_Card_Project> pip install -r requirements.txt
Using cached kiwisolver-1.5.0-cp312-cp312-win_amd64.whl (73 kB)
Using cached markupsafe-3.0.3-cp312-cp312-win_amd64.whl (15 kB)
Using cached matplotlib-3.11.0-cp312-cp312-win_amd64.whl (9.3 MB)
Using cached narwhals-2.22.1-py3-none-any.whl (454 kB)
Using cached numpy-2.5.0-cp312-cp312-win_amd64.whl (12.4 MB)
Using cached packaging-26.2-py3-none-any.whl (100 kB)
Using cached pandas-3.0.3-cp312-cp312-win_amd64.whl (9.8 MB)
Using cached pillow-12.2.0-cp312-cp312-win_amd64.whl (7.1 MB)
Using cached pyparsing-3.3.2-py3-none-any.whl (122 kB)
Using cached python_dateutil-2.9.0.post0-py2.py3-none-any.whl (229 kB)
Using cached scikit_learn-1.9.0-cp312-cp312-win_amd64.whl (8.2 MB)
Using cached scipy-1.18.0-cp312-cp312-win_amd64.whl (36.6 MB)
Using cached seaborn-0.13.2-py3-none-any.whl (294 kB)
Using cached six-1.17.0-py2.py3-none-any.whl (11 kB)
Using cached sklearn_compat-0.1.6-py3-none-any.whl (22 kB)
Using cached threadpoolctl-3.6.0-py3-none-any.whl (18 kB)
Using cached tzdata-2026.2-py2.py3-none-any.whl (349 kB)
Using cached werkzeug-3.1.8-py3-none-any.whl (226 kB)
Installing collected packages: tzdata, threadpoolctl, six, pyparsing, pillow, packaging, numpy, narwhals, MarkupSafe, kiwisolver, joblib, itsdangerous, fonttools,
cycler, colorama, blinker, Werkzeug, scipy, python-dateutil, Jinja2, contourpy, click, scikit-learn, pandas, matplotlib, Flask, sklearn-compat, seaborn, imbalanced-learn
Successfully installed Flask-3.1.3 Jinja2-3.1.6 MarkupSafe-3.0.3 Werkzeug-3.1.8 blinker-1.9.0 click-8.4.2 colorama-0.4.6 contourpy-1.3.3 cycler-0.12.1 fonttools-
4.63.0 imbalanced-learn-0.14.2 itsdangerous-2.2.0 joblib-1.5.3 kiwisolver-1.5.0 matplotlib-3.11.0 narwhals-2.22.1 numpy-2.5.0 packaging-26.2 pandas-3.0.3 pillow-12.2.0 pyparsing-3.3.2 python-dateutil-2.9.0.post0 scikit-learn-1.9.0 scipy-1.18.0 seaborn-0.13.2 six-1.17.0 sklearn-compat-0.1.6 threadpoolctl-3.6.0 tzdata-2026.2

[notice] A new release of pip is available: 25.0.1 -> 26.1.2
[notice] To update, run: python.exe -m pip install --upgrade pip
(.venv) PS C:\Users\johns\Downloads\Credit_Card_Project> █
```

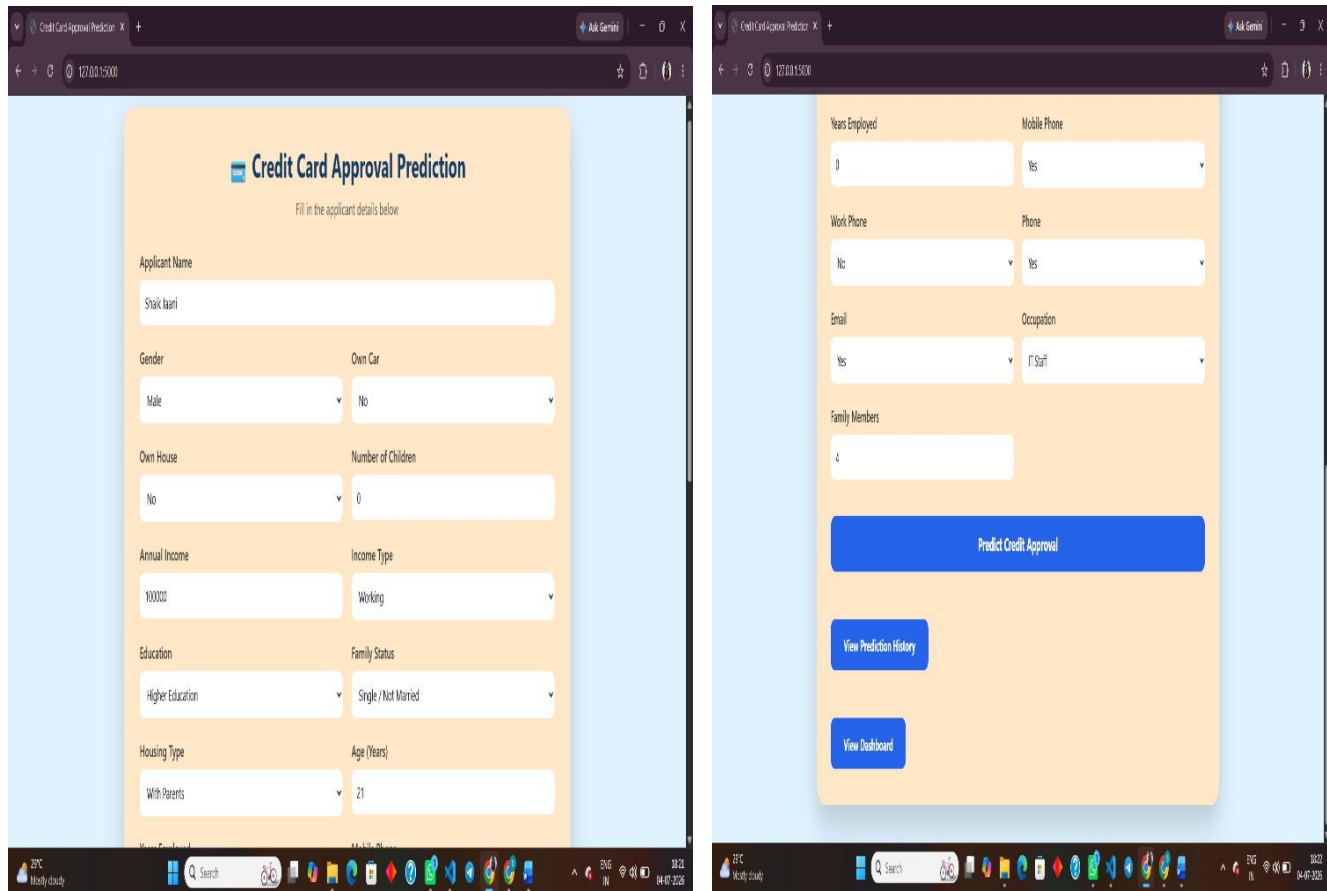
Verify Installation

Run the Flask application.

`python app.py`

Open the application in a browser.

<http://127.0.0.1:5000>



Credit Card Approval Prediction
Fill in the applicant details below

Applicant Name
Shakilani

Gender: Male
Own Car: No

Own House: No
Number of Children: 0

Annual Income: 100000
Income Type: Working

Education: Higher Education
Family Status: Single / Not Married

Housing Type: With Parents
Age (Years): 21

Years Employed: 0
Mobile Phone: Yes

Work Phone: No
Phone: Yes

Email: Yes
Occupation: IT Staff

Family Members: 4

Predict Credit Approval

[View Prediction History](#)

[View Dashboard](#)

Milestone 2: Core Functionalities Development

In this milestone, the core functionalities of the Credit Card Approval Prediction System are implemented. The system integrates machine learning with a Flask-based web application to provide secure login authentication, applicant data processing, real-time credit card approval prediction, prediction history management, and dashboard analytics. The trained Random Forest model is connected to the web application to generate accurate predictions based on applicant information.

Activity 2.1: Develop Core Prediction Features

Implement the major functionalities of the Credit Card Approval Prediction System.

Applicant Data Collection

- Design a web-based form to collect applicant personal, financial, and employment details.
- Validate the entered information before processing the prediction.
- Convert user inputs into the format required by the trained machine learning model.

127.0.0.1:5000

Credit Card Approval Prediction

Fill in the applicant details below

Applicant Name Shaik	
Gender Male	Own Car No
Own House No	Number of Children 0
Annual Income 10000	Income Type Student
Education Secondary / Secondary Special	Family Status Single / Not Married
Housing Type Rented Apartment	Age (Years) 21

Credit Card Approval Prediction

- Load the trained **Random Forest** model (model.pkl).
- Accept applicant information from the Flask application.
- Predict whether the applicant is a **Good Customer (Approved)** or **Bad Customer (Rejected)**.
- Display the prediction result with the confidence score.

Prediction Result

Applicant Details

Applicant Name : Shaik

Prediction : Good Customer

Confidence : 69.26 %

Prediction Date : 05-07-2026

Prediction Time : 03:53 PM



Credit Card Approved

Congratulations! **Shaik** is predicted to be a **Good Customer** with **69.26%** confidence.

[Predict Another Applicant](#)

Prediction History

- Store every prediction in the SQLite database.
- Display previously generated prediction records on the Prediction History page.
- Allow users to review prediction history whenever required.

Prediction History

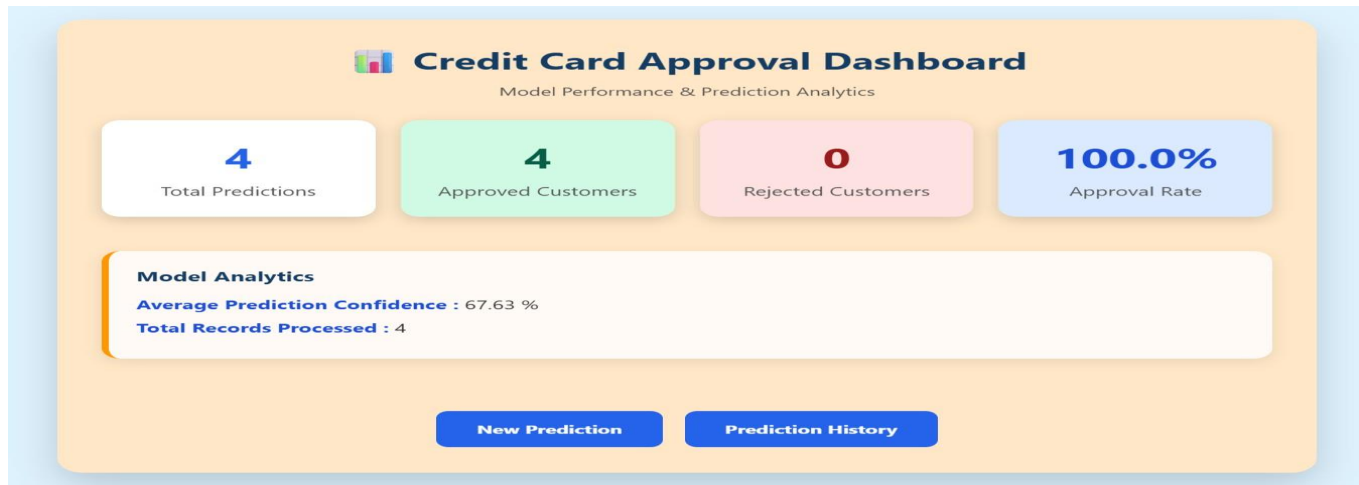
All Saved Credit Card Predictions

ID	Applicant Name	Prediction	Confidence	Date	Time
4	Shaik	Good Customer	69.26 %	05-07-2026	03:53 PM
3	Shaik	Good Customer	69.26 %	05-07-2026	03:51 PM
2	Shaik Jaani	Good Customer	61.65 %	04-07-2026	06:22 PM
1	Jaani	Good Customer	70.36 %	04-07-2026	12:29 PM

[← Back to Prediction](#)

Dashboard Analytics

- Display total predictions.
- Display approved customers.
- Display rejected customers.
- Display approval rate.
- Display average prediction confidence.



Activity 2.2: Implement the Flask Backend

Define Routes in Flask

- Create routes for Login, Home, Prediction, Prediction History, Dashboard, and Logout.
- Load the trained Random Forest model when the application starts.
- Render HTML templates using Flask.

```
& c:\Users\johns\Downloads\Credit_Card_Project\.venv\Scripts\Activate.ps1)
(.venv) PS C:\Users\johns\Downloads\Credit_Card_Project\5. Project Development Phase\Source Code> python app.py
* Serving Flask app 'app'
* Debug mode: on
WARNING: This is a development server. Do not use it in a production deployment. Use a production WSGI server instead.
* Running on http://127.0.0.1:5000
Press CTRL+C to quit
* Restarting with stat
* Debugger is active!
* Debugger PIN: 291-358-805
127.0.0.1 - - [05/Jul/2026 15:50:40] "GET / HTTP/1.1" 200 -
127.0.0.1 - - [05/Jul/2026 15:50:41] "GET /static/style.css HTTP/1.1" 200 -
127.0.0.1 - - [05/Jul/2026 15:50:41] "GET /favicon.ico HTTP/1.1" 404 -
C:\Users\johns\Downloads\Credit_Card_Project\.venv\Lib\site-packages\sklearn\utils\validation.py:2827: UserWarning: X does not have valid feature names, but RandomForestClassifier was fitted with feature names
  warnings.warn(
C:\Users\johns\Downloads\Credit_Card_Project\.venv\Lib\site-packages\sklearn\utils\validation.py:2827: UserWarning: X does not have valid feature names, but RandomForestClassifier was fitted with feature names
  warnings.warn(
127.0.0.1 - - [05/Jul/2026 15:51:55] "POST /predict HTTP/1.1" 200 -
127.0.0.1 - - [05/Jul/2026 15:51:55] "GET /static/style.css HTTP/1.1" 304 -
C:\Users\johns\Downloads\Credit_Card_Project\.venv\Lib\site-packages\sklearn\utils\validation.py:2827: UserWarning: X does not have valid feature names, but RandomForestClassifier was fitted with feature names
  warnings.warn(
C:\Users\johns\Downloads\Credit_Card_Project\.venv\Lib\site-packages\sklearn\utils\validation.py:2827: UserWarning: X does not have valid feature names, but RandomForestClassifier was fitted with feature names
  warnings.warn(
127.0.0.1 - - [05/Jul/2026 15:53:26] "POST /predict HTTP/1.1" 200 -
127.0.0.1 - - [05/Jul/2026 15:53:26] "GET /static/style.css HTTP/1.1" 304 -
127.0.0.1 - - [05/Jul/2026 15:55:35] "GET / HTTP/1.1" 200 -
127.0.0.1 - - [05/Jul/2026 15:55:35] "GET /static/style.css HTTP/1.1" 304 -
```

Process User Input

- Create an applicant input form using HTML and CSS.
- Receive user input through Flask using request.form.
- Validate the applicant details before prediction.
- Convert age and employment years into the required format for the machine learning model.

Machine Learning Prediction

- Pass the processed applicant data to the trained Random Forest model.
- Generate the prediction and calculate the confidence score.
- Display the prediction result on the Result page.

SQLite Database Integration

- Connect the Flask application with the SQLite database.
- Store applicant name, prediction result, confidence score, prediction date, and prediction time.
- Retrieve stored records for the Prediction History page.

Dashboard Analytics

- Retrieve prediction records from the SQLite database.
- Calculate statistics such as total predictions, approvals, rejections, approval rate, and average confidence.
- Display the analytics on the Dashboard page.

Milestone 3: App.py Development

Milestone 3 focuses on developing and refining the app.py file, which serves as the core component of the Credit Card Approval Prediction System. The Flask application manages user authentication, applicant data processing, machine learning prediction, SQLite database operations, prediction history management, and dashboard analytics. It acts as the bridge between the frontend interface, the trained Random Forest model, and the SQLite database to provide real-time credit card approval predictions.

Activity 3.1: Writing the Main Application Logic in app.py

Define the Core Routes in app.py

Create the primary Flask routes responsible for the application's functionality.

Home Route (/)

- Displays the Login page or Home page.
- Authenticates users before allowing access to the prediction system.

```
# =====
# HOME PAGE
# =====

@app.route("/")
def home():
    return render_template("index.html")
```

Prediction Route (/predict)

- Accept applicant details submitted through the HTML form.
- Validate the user input.
- Convert age and employment years into the required format.
- Pass the processed data to the trained Random Forest model.
- Generate the prediction and confidence score.
- Display the prediction result.

```
# -----
# PREDICTION
# -----

@app.route("/predict", methods=["POST"])
def predict():
    try:
        # -----
        # APPLICANT NAME
        # -----

        applicant_name = request.form["APPLICANT_NAME"]

        # -----
        # COLLECT INPUT FEATURES
        # -----

        features = [
            float(request.form["CODE_GENDER"]),
            float(request.form["FLAG_OWN_CAR"]),
            float(request.form["FLAG_OWN_REALTY"]),
            float(request.form["CNT_CHILDREN"]),
            float(request.form["CNT_INCOME_TOTAL"]),
            float(request.form["NAME_INCOME_TYPE"]),
            float(request.form["NAME_EDUCATION_TYPE"]),
            float(request.form["NAME_FAMILY_STATUS"]),
            float(request.form["NAME_HOUSING_TYPE"]),
            float(request.form["DAYS_BIRTH"]),
            float(request.form["DAYS_EMPLOYED"]),
            float(request.form["FLAG_MOBIL"]),
            float(request.form["FLAG_WORK_PHONE"]),
            float(request.form["FLAG_PHONE"]),
            float(request.form["FLAG_EMAIL"]),
            float(request.form["OCCUPATION_TYPE"]),
            float(request.form["CNT_FAM_MEMBERS"])
        ]

        # -----
        # CONVERT INPUT TO DATAFRAME
        # (Removes sklearn feature-name warning)
        # -----

        columns = [
            "CODE_GENDER",
            "FLAG_OWN_CAR",
            "FLAG_OWN_REALTY",
            "CNT_CHILDREN",
            "CNT_INCOME_TOTAL",
            "NAME_INCOME_TYPE",
            "NAME_EDUCATION_TYPE",
            "NAME_FAMILY_STATUS",
            "NAME_HOUSING_TYPE",
            "DAYS_BIRTH",
            "DAYS_EMPLOYED",
            "FLAG_MOBIL",
            "FLAG_WORK_PHONE",
            "FLAG_PHONE",
            "FLAG_EMAIL",
            "OCCUPATION_TYPE",
            "CNT_FAM_MEMBERS"
        ]

        features = pd.DataFrame([features], columns=columns)

        # -----
        # MODEL PREDICTION
        # -----

        prediction = model.predict(features)[0]
        probability = model.predict_proba(features)

        confidence = round(np.max(probability) * 100, 2)

        # -----
        # PREDICTION RESULT
        # -----

        if prediction == 0:
            result = "Good Customer"
            approval = "Credit Card Approved"
            icon = "✅"
        else:
            result = "Bad Customer"
            approval = "Credit Card Rejected"
```

```
            icon = "❌"

        # -----
        # DATE & TIME
        # -----

        current_date = datetime.now().strftime("%d-%m-%Y")
        current_time = datetime.now().strftime("%I:%M %p")

        # SAVE PREDICTION TO SQLITE DATABASE
        # -----

        connection = sqlite3.connect("credit_card.db")
        cursor = connection.cursor()

        cursor.execute(
            """
            INSERT INTO predictions
            (
                applicant_name,
                prediction,
                confidence,
                prediction_date,
                prediction_time
            )
            VALUES (?, ?, ?, ?, ?)
            """
        )

        connection.commit()

        connection.close()

        # -----
        # DISPLAY RESULT PAGE
        # -----

        return render_template(
            "result.html",
            applicant_name=applicant_name,
            prediction=result,
            approval=approval,
            icon=icon,
            confidence=confidence,
            prediction_date=current_date,
            prediction_time=current_time
        )

    except Exception as e:
        return render_template(
            "result.html",
            prediction=f"Error : {str(e)}"
        )
```

History Route (/history)

- Connect to the SQLite database.
- Retrieve all stored prediction records.
- Display the records in the Prediction History page.

```
212 # PREDICTION HISTORY
213 # =====
214
215 @app.route("/history")
216 def history():
217
218     connection = sqlite3.connect("credit_card.db")
219
220     cursor = connection.cursor()
221
222     cursor.execute("""
223
224     SELECT
225         id,
226         applicant_name,
227         prediction,
228         confidence,
229         prediction_date,
230         prediction_time
231
232     FROM predictions
233
234     ORDER BY id DESC
235
236     """)
237
238     history_data = cursor.fetchall()
239
240     connection.close()
241
242     return render_template(
243         "history.html",
244         history=history_data
245     )
246
247
248
```

Dashboard Route (/dashboard)

- Read prediction records from the SQLite database.
- Calculate total predictions, approved customers, rejected customers, approval rate, and average confidence.
- Display analytics on the Dashboard page.

```
249 # =====
250 # DASHBOARD
251 # =====
252
253 @app.route("/dashboard")
254 def dashboard():
255
256     connection = sqlite3.connect("credit_card.db")
257
258     cursor = connection.cursor()
259
260     # -----
261     # TOTAL PREDICTIONS
262     # -----
263
264     cursor.execute("SELECT COUNT(*) FROM predictions")
265
266     total = cursor.fetchone()[0]
267
268     # -----
269     # APPROVED CUSTOMERS
270     # -----
271
272     cursor.execute(
273         "SELECT COUNT(*) FROM predictions WHERE prediction='Good Customer'"
274     )
275
276
277     approved = cursor.fetchone()[0]
278
279
280     # -----
281     # REJECTED CUSTOMERS
282     # -----
283
284     cursor.execute(
```

Process User Input

- Retrieve applicant information using request.form.
- Validate mandatory fields before prediction.
- Convert age and years employed into the required machine learning format.
- Create the feature vector required by the Random Forest model.

```
# -----  
# APPLICANT NAME  
# -----  
  
applicant_name = request.form["APPLICANT_NAME"]  
  
# -----  
# COLLECT INPUT FEATURES  
# -----  
  
features = [  
    float(request.form["CODE_GENDER"]),  
    float(request.form["FLAG_OWN_CAR"]),  
    float(request.form["FLAG_OWN_REALTY"]),  
    float(request.form["CNT_CHILDREN"]),  
    float(request.form["AMT_INCOME_TOTAL"]),  
    float(request.form["NAME_INCOME_TYPE"]),  
    float(request.form["NAME_EDUCATION_TYPE"]),  
    float(request.form["NAME_FAMILY_STATUS"]),  
    float(request.form["NAME_HOUSING_TYPE"]),  
    float(request.form["DAYS_BIRTH"]),  
    float(request.form["DAYS_EMPLOYED"]),  
    float(request.form["FLAG_MOBIL"]),  
    float(request.form["FLAG_WORK_PHONE"]),  
    float(request.form["FLAG_PHONE"]),  
    float(request.form["FLAG_EMAIL"]),  
    float(request.form["OCCUPATION_TYPE"]),  
    float(request.form["CNT_FAM_MEMBERS"])  
]
```

Machine Learning Prediction

- Load the trained Random Forest model (model.pkl).
- Generate the prediction using model.predict().
- Calculate the confidence score using model.predict_proba().
- Determine whether the applicant is a Good Customer or Bad Customer.

```
def predict():

    columns = [

        "CODE_GENDER",
        "FLAG_OWN_CAR",
        "FLAG_OWN_REALTY",
        "CNT_CHILDREN",
        "AMT_INCOME_TOTAL",
        "NAME_INCOME_TYPE",
        "NAME_EDUCATION_TYPE",
        "NAME_FAMILY_STATUS",
        "NAME_HOUSING_TYPE",
        "DAYS_BIRTH",
        "DAYS_EMPLOYED",
        "FLAG_MOBIL",
        "FLAG_WORK_PHONE",
        "FLAG_PHONE",
        "FLAG_EMAIL",
        "OCCUPATION_TYPE",
        "CNT_FAM_MEMBERS"

    ]

    features = pd.DataFrame([features], columns=columns)

    # -----
    # MODEL PREDICTION
    # -----

    prediction = model.predict(features)[0]

    probability = model.predict_proba(features)

    confidence = round(np.max(probability) * 100, 2)
```

SQLite Database Integration

- Establish a connection with the SQLite database.
- Store applicant name, prediction result, confidence score, prediction date, and prediction time.
- Commit the transaction and close the database connection.


```
def predict():

    current_date = datetime.now().strftime("%d-%m-%Y")

    current_time = datetime.now().strftime("%I:%M %p")
    # -----
    # SAVE PREDICTION TO SQLITE DATABASE
    # -----

    connection = sqlite3.connect("credit_card.db")

    cursor = connection.cursor()

    cursor.execute(
        """
        INSERT INTO predictions
        (
            applicant_name,
            prediction,
            confidence,
            prediction_date,
            prediction_time
        )
        VALUES (?, ?, ?, ?, ?)
        """,
        (
            applicant_name,
            result,
            confidence,
            current_date,
            current_time
        )
    )

    connection.commit()

    connection.close()
```

Dashboard Analytics

- Retrieve prediction records from the SQLite database.
- Calculate statistics including:
 - Total Predictions
 - Approved Customers
 - Rejected Customers
 - Approval Rate
 - Average Confidence Score
- Pass the calculated values to the dashboard template.
-

```
@app.route("/dashboard")
def dashboard():

    connection = sqlite3.connect("credit_card.db")

    cursor = connection.cursor()

    # -----
    # TOTAL PREDICTIONS
    # -----

    cursor.execute("SELECT COUNT(*) FROM predictions")

    total = cursor.fetchone()[0]

    # -----
    # APPROVED CUSTOMERS
    # -----

    cursor.execute(
        "SELECT COUNT(*) FROM predictions WHERE prediction='Good Customer'"
    )

    approved = cursor.fetchone()[0]

    # -----
    # REJECTED CUSTOMERS
    # -----

    cursor.execute(
        "SELECT COUNT(*) FROM predictions WHERE prediction='Bad Customer'"
    )
```

```
)

rejected = cursor.fetchone()[0]

# -----
# AVERAGE CONFIDENCE
# -----

cursor.execute(
    "SELECT AVG(confidence) FROM predictions"
)

average_confidence = cursor.fetchone()[0]

if average_confidence is None:
    average_confidence = 0

average_confidence = round(average_confidence, 2)

# -----
# APPROVAL RATE
# -----

approval_rate = 0

if total > 0:
    approval_rate = round((approved / total) * 100, 2)

connection.close()
```

Error Handling

- Handle invalid user input.
- Handle database connection errors.
- Handle model prediction exceptions.
- Display meaningful error messages to the user.

```

Project Development Phase / Source Code / app.py / predict
41 def predict():
42
43     try:
44
45         # -----
46         # APPLICANT NAME
47         # -----
48
49         applicant_name = request.form["APPLICANT_NAME"]
50
51         # -----
52         # COLLECT INPUT FEATURES
53         # -----
54
55         features = [
56
57             float(request.form["CODE_GENDER"]),
58             float(request.form["FLAG_OWN_CAR"]),
59             float(request.form["FLAG_OWN_REALTY"]),
60             float(request.form["CNT_CHILDREN"]),
61             float(request.form["AMT_INCOME_TOTAL"]),
62             float(request.form["NAME_INCOME_TYPE"]),
63             float(request.form["NAME_EDUCATION_TYPE"]),
64             float(request.form["NAME_FAMILY_STATUS"]),
65             float(request.form["NAME_HOUSING_TYPE"]),
66             float(request.form["DAYS_BIRTH"]),
67             float(request.form["DAYS_EMPLOYED"]),
68             float(request.form["FLAG_MOBIL"]),
69             float(request.form["FLAG_WORK_PHONE"]),
70             float(request.form["FLAG_PHONE"]),
71             float(request.form["FLAG_EMAIL"]),
72             float(request.form["OCCUPATION_TYPE"]),
73             float(request.form["CNT_FAM_MEMBERS"])
74
75         ]
76

```

```

41 def predict():
175
176     # -----
177     # DISPLAY RESULT PAGE
178     # -----
179
180     return render_template(
181
182         "result.html",
183
184         applicant_name=applicant_name,
185
186         prediction=result,
187
188         approval=approval,
189
190         icon=icon,
191
192         confidence=confidence,
193
194         prediction_date=current_date,
195
196         prediction_time=current_time
197
198     )
199
200     except Exception as e:
201
202         return render_template(
203
204             "result.html",
205
206             prediction=f"Error : {str(e)}"
207
208         )
209
210

```

Milestone 4: Frontend Development

Milestone 4 focuses on designing and developing a responsive, user-friendly, and visually appealing web interface for the **Credit Card Approval Prediction System**. The frontend is developed using **HTML**, **CSS**, and **JavaScript** to provide an interactive experience for users. It includes pages for applicant data entry, prediction results, prediction history, dashboard analytics, and user authentication. The interface is designed to ensure smooth navigation and an intuitive user experience across different devices.

Activity 4.1: Designing and Developing the User Interface

Set Up the Base HTML Structure

- Develop the main **index.html** page to collect applicant details required for credit card approval prediction.
- Create separate HTML pages for **Prediction Result, Prediction History, Dashboard, and Login.**
- Use semantic HTML elements to organize the application structure and improve readability.
- Design navigation buttons to easily access Prediction History and Dashboard pages.



The screenshot shows a web browser window with the address 127.0.0.1:5000. The page title is "Credit Card Approval Prediction" with a subtitle "Fill in the applicant details below". The form contains the following fields:

- Applicant Name: Text input with value "Shaik"
- Gender: Dropdown menu with value "Male"
- Own Car: Dropdown menu with value "No"
- Own House: Dropdown menu with value "No"
- Number of Children: Text input with value "0"
- Annual Income: Text input with value "10000"
- Income Type: Dropdown menu with value "Student"
- Education: Dropdown menu with value "Secondary / Secondary Special"
- Family Status: Dropdown menu with value "Single / Not Married"
- Housing Type: Dropdown menu with value "Rented Apartment"
- Age (Years): Text input with value "21"

Design a Responsive Layout Using CSS

- Create a CSS stylesheet (style.css) to provide a clean, responsive, and modern user interface.
- Use **CSS Grid** and **Flexbox** layouts to organize input fields, buttons, and result sections.
- Apply responsive design using media queries to ensure compatibility with desktop, tablet, and mobile devices.
- Add hover effects, rounded cards, shadows, and animations to enhance the user experience.

```

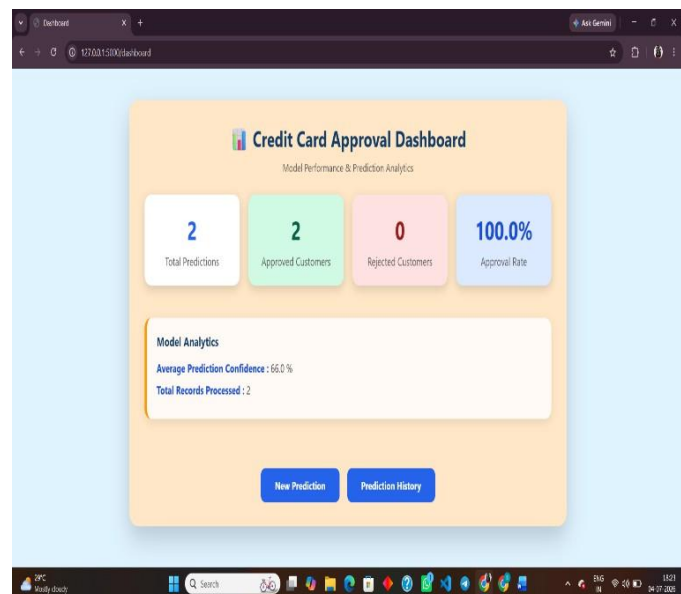
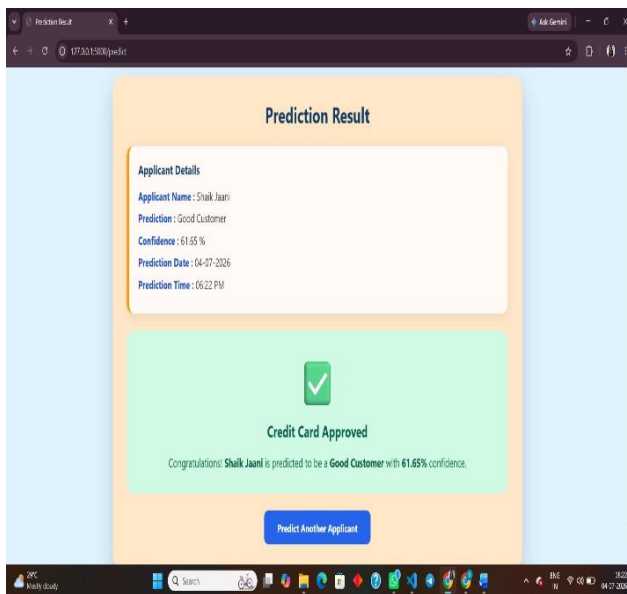
41  def predict():
175
176      # -----
177      #  DISPLAY RESULT PAGE
178      #  -----
179
180      return render_template(
181
182          "result.html",
183          applicant_name=applicant_name,
184          prediction=result,
185          approval=approval,
186          icon=icon,
187          confidence=confidence,
188          prediction_date=current_date,
189          prediction_time=current_time
190      )
191
192  except Exception as e:
193
194      return render_template(
195
196          "result.html",
197          prediction=f"Error : {str(e)}"
198      )
199
200
201
202
203
204
205
206
207
208
209
210

```

Create Separate UI Components

Develop separate user interface components for each module of the application:

- **Applicant Details Form:** Collect applicant personal, financial, and employment information.
- **Prediction Result Card:** Display the applicant name, prediction result, confidence score, date, and time.
- **Prediction History Table:** Display all previous prediction records stored in the SQLite database.
- **Dashboard Cards:** Display total predictions, approved customers, rejected customers, approval rate, and average confidence.



Activity 4.2: Creating Dynamic Functionality Using JavaScript

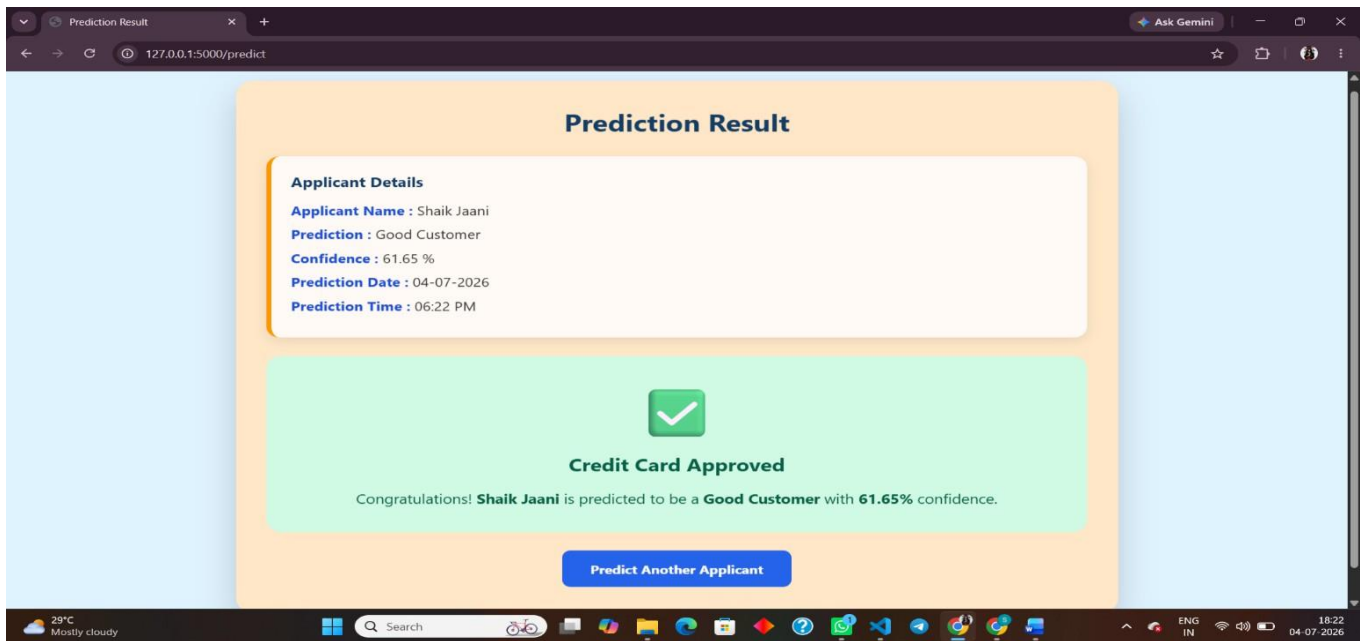
Handle Form Submission

- Capture applicant information entered by the user.
- Convert **Age** and **Years Employed** into the required format (**Days Birth** and **Days Employed**) before submitting the form.
- Submit the applicant data to the Flask backend for prediction.

```
<script>
document.querySelector("form").addEventListener("submit",function(){
  let age=document.getElementById("age").value;
  document.getElementById("days_birth").value=-(age*365);
  let exp=document.getElementById("experience").value;
  document.getElementById("days_employed").value=-(exp*365);
});
</script>
</body>
</html>
```

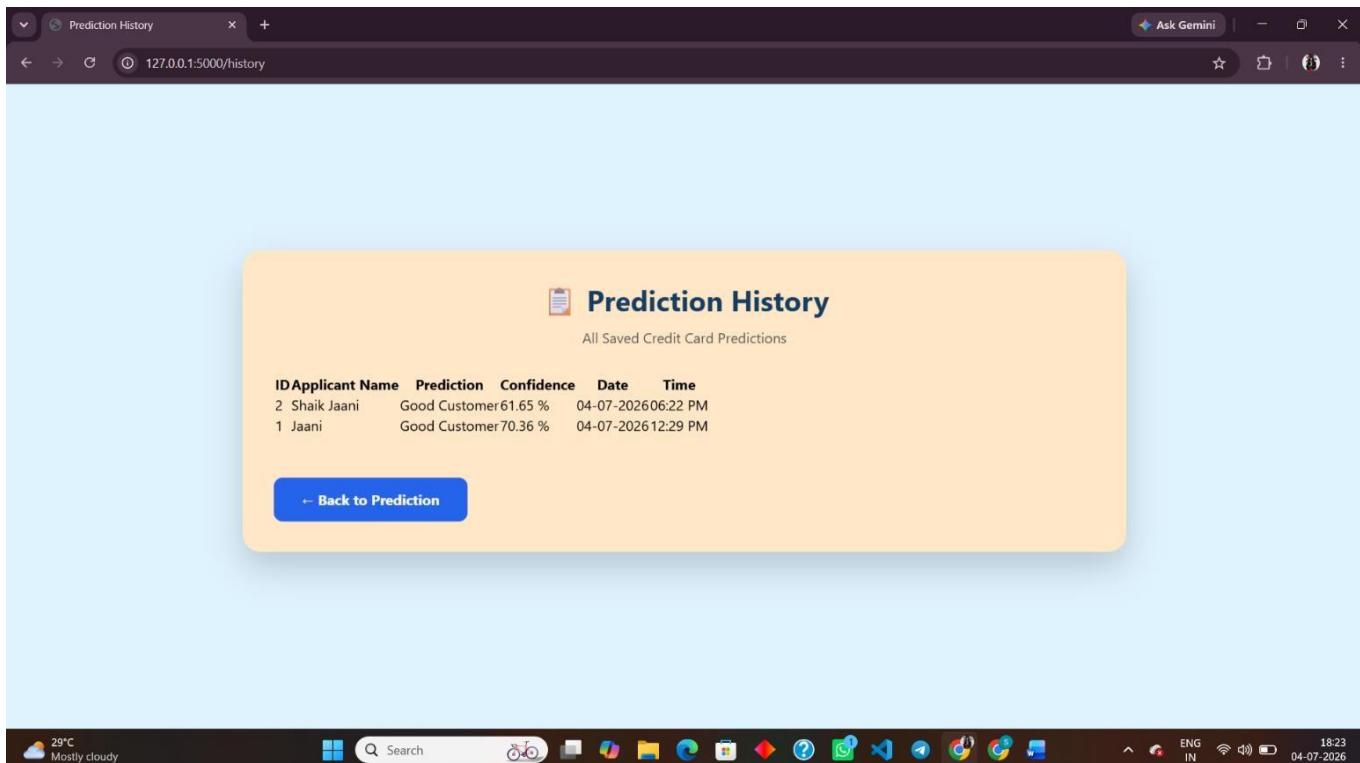

Display Prediction Results

- Display the applicant's credit card approval result.
- Show whether the applicant is a **Good Customer** or **Bad Customer**.
- Display the confidence score, prediction date, and prediction time.
- Provide navigation buttons to access Prediction History and Dashboard.



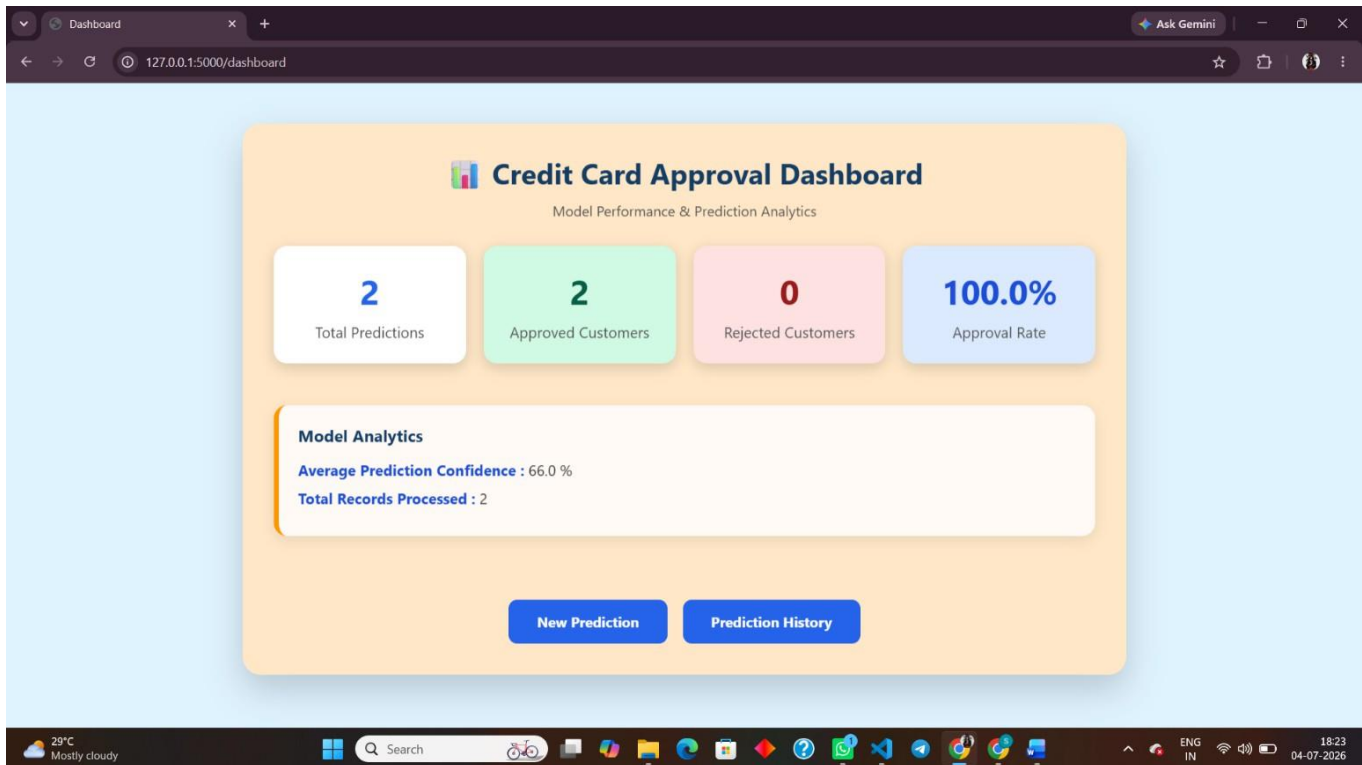
Display Stored Prediction History

- Retrieve prediction records from the SQLite database.
- Display all previous prediction results in a structured table.
- Show applicant name, prediction result, confidence score, date, and time.



Display Dashboard Analytics

- Display the total number of predictions.
- Display the total approved customers.
- Display the total rejected customers.
- Display the approval rate.
- Display the average prediction confidence.



Milestone 5: Deployment

Milestone 5 focuses on deploying the **Credit Card Approval Prediction System** in a local development environment using the Flask framework. The application is configured with all required Python libraries and executed through the Flask development server. After deployment, the system is thoroughly tested to ensure that all modules, including user authentication, applicant data processing, machine learning prediction, SQLite database integration, prediction history, and dashboard analytics, function correctly.

Activity 5.1: Preparing the Application for Local Deployment

Set Up a Virtual Environment

- Open **Visual Studio Code**.
- Open the integrated terminal.
- Create and activate a Python virtual environment.

```
python -m venv .venv
```

```
.venv\Scripts\activate
```

```
• PS C:\Users\johns\Downloads\Credit_Card_Project> python -m venv .venv
• PS C:\Users\johns\Downloads\Credit_Card_Project> .\.venv\Scripts\activate
○ (.venv) PS C:\Users\johns\Downloads\Credit_Card_Project> |
```

Install Required Dependencies

Install all required project libraries using the requirements file.

pip install -r requirements.txt

or

pip install flask numpy pandas scikit-learn matplotlib seaborn

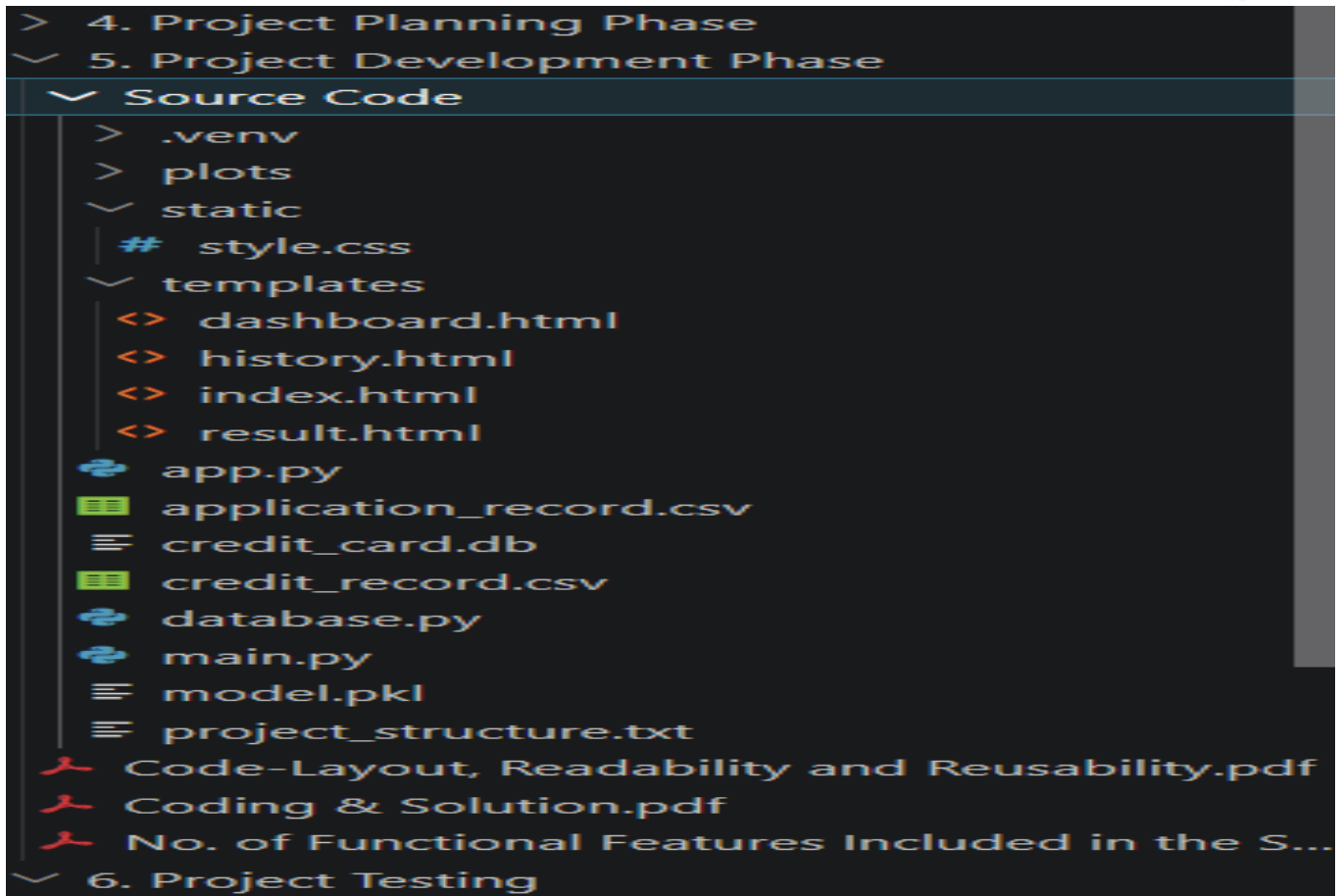
```
(.venv) PS C:\Users\johns\Downloads\Credit_Card_Project> pip install -r requirements.txt
Using cached kiwisolver-1.5.0-cp312-cp312-win_amd64.whl (73 kB)
Using cached markupsafe-3.0.3-cp312-cp312-win_amd64.whl (15 kB)
Using cached matplotlib-3.11.0-cp312-cp312-win_amd64.whl (9.3 MB)
Using cached narwhals-2.22.1-py3-none-any.whl (454 kB)
Using cached numpy-2.5.0-cp312-cp312-win_amd64.whl (12.4 MB)
Using cached packaging-26.2-py3-none-any.whl (100 kB)
Using cached pandas-3.0.3-cp312-cp312-win_amd64.whl (9.8 MB)
Using cached pillow-12.2.0-cp312-cp312-win_amd64.whl (7.1 MB)
Using cached pyparsing-3.3.2-py3-none-any.whl (122 kB)
Using cached python_dateutil-2.9.0.post0-py2.py3-none-any.whl (229 kB)
Using cached scikit_learn-1.9.0-cp312-cp312-win_amd64.whl (8.2 MB)
Using cached scipy-1.18.0-cp312-cp312-win_amd64.whl (36.6 MB)
Using cached seaborn-0.13.2-py3-none-any.whl (294 kB)
Using cached six-1.17.0-py2.py3-none-any.whl (11 kB)
Using cached sklearn_compat-0.1.6-py3-none-any.whl (22 kB)
Using cached threadpoolctl-3.6.0-py3-none-any.whl (18 kB)
Using cached tzdata-2026.2-py2.py3-none-any.whl (349 kB)
Using cached werkzeug-3.1.8-py3-none-any.whl (226 kB)
Installing collected packages: tzdata, threadpoolctl, six, pyparsing, pillow, packaging, numpy, narwhals, MarkupSafe, kiwisolver, joblib, itsdangerous, fonttools, cycler, colorama, blinker, Werkzeug, scipy, python-dateutil, Jinja2, contourpy, click, scikit-learn, pandas, matplotlib, Flask, sklearn-compat, seaborn, imbalanced-learn
Successfully installed Flask-3.1.3 Jinja2-3.1.6 MarkupSafe-3.0.3 Werkzeug-3.1.8 blinker-1.9.0 click-8.4.2 colorama-0.4.6 contourpy-1.3.3 cycler-0.12.1 fonttools-4.63.0 imbalanced-learn-0.14.2 itsdangerous-2.2.0 joblib-1.5.3 kiwisolver-1.5.0 matplotlib-3.11.0 narwhals-2.22.1 numpy-2.5.0 packaging-26.2 pandas-3.0.3 pillow-12.2.0 pyparsing-3.3.2 python-dateutil-2.9.0.post0 scikit-learn-1.9.0 scipy-1.18.0 seaborn-0.13.2 six-1.17.0 sklearn-compat-0.1.6 threadpoolctl-3.6.0 tzdata-2026.2

[notice] A new release of pip is available: 25.0.1 -> 26.1.2
[notice] To update, run: python.exe -m pip install --upgrade pip
(.venv) PS C:\Users\johns\Downloads\Credit_Card_Project> |
```

Verify Project Files

Ensure that all required files are available in the project directory, including:

- app.py
- model.pkl
- credit_card.db
- templates/
- static/
- requirements.txt

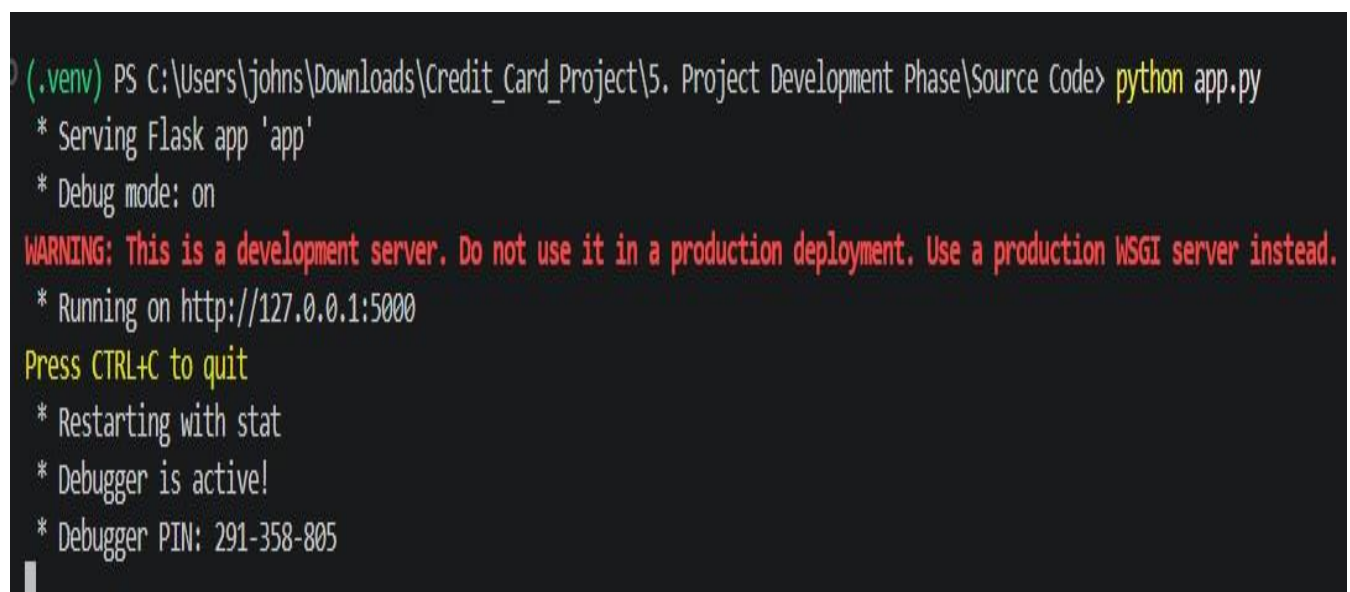


Activity 5.2: Local Testing and Verification

Start the Flask Development Server

Run the application using the following command:

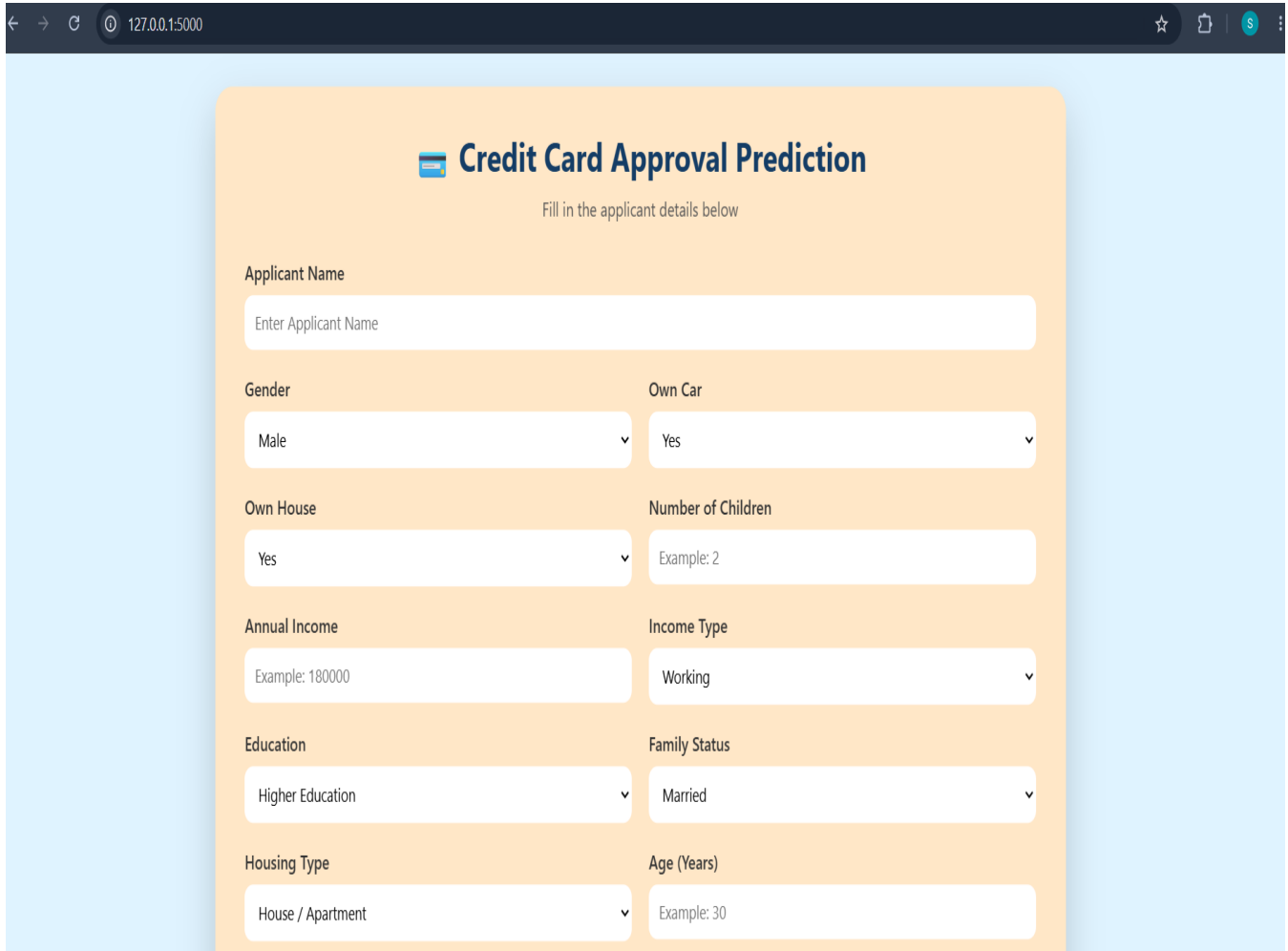
```
python app.py
```



Access the Application

Open a web browser and navigate to:

<http://127.0.0.1:5000/>



The screenshot shows a web browser window with the address bar displaying `127.0.0.1:5000`. The application interface has a light blue background. In the center, there is an orange rounded rectangle containing the title "Credit Card Approval Prediction" with a credit card icon to its left. Below the title, it says "Fill in the applicant details below". The form consists of several input fields and dropdown menus arranged in two columns:

- Applicant Name:** A text input field with the placeholder "Enter Applicant Name".
- Gender:** A dropdown menu with "Male" selected.
- Own Car:** A dropdown menu with "Yes" selected.
- Own House:** A dropdown menu with "Yes" selected.
- Number of Children:** A text input field with the placeholder "Example: 2".
- Annual Income:** A text input field with the placeholder "Example: 180000".
- Income Type:** A dropdown menu with "Working" selected.
- Education:** A dropdown menu with "Higher Education" selected.
- Family Status:** A dropdown menu with "Married" selected.
- Housing Type:** A dropdown menu with "House / Apartment" selected.
- Age (Years):** A text input field with the placeholder "Example: 30".

Verify Application Functionalities

Test all major modules of the application:

- Login Authentication (if implemented)
- Applicant Data Entry
- Credit Card Approval Prediction
- Prediction Confidence Score
- Prediction History
- Dashboard Analytics
- SQLite Database Storage

Prediction Result

Applicant Details
Applicant Name : Shaik
Prediction : Good Customer
Confidence : 69.26 %
Prediction Date : 05-07-2026
Prediction Time : 03:53 PM



Credit Card Approved

Congratulations! **Shaik** is predicted to be a **Good Customer** with **69.26%** confidence.

[Predict Another Applicant](#)

Prediction History

All Saved Credit Card Predictions

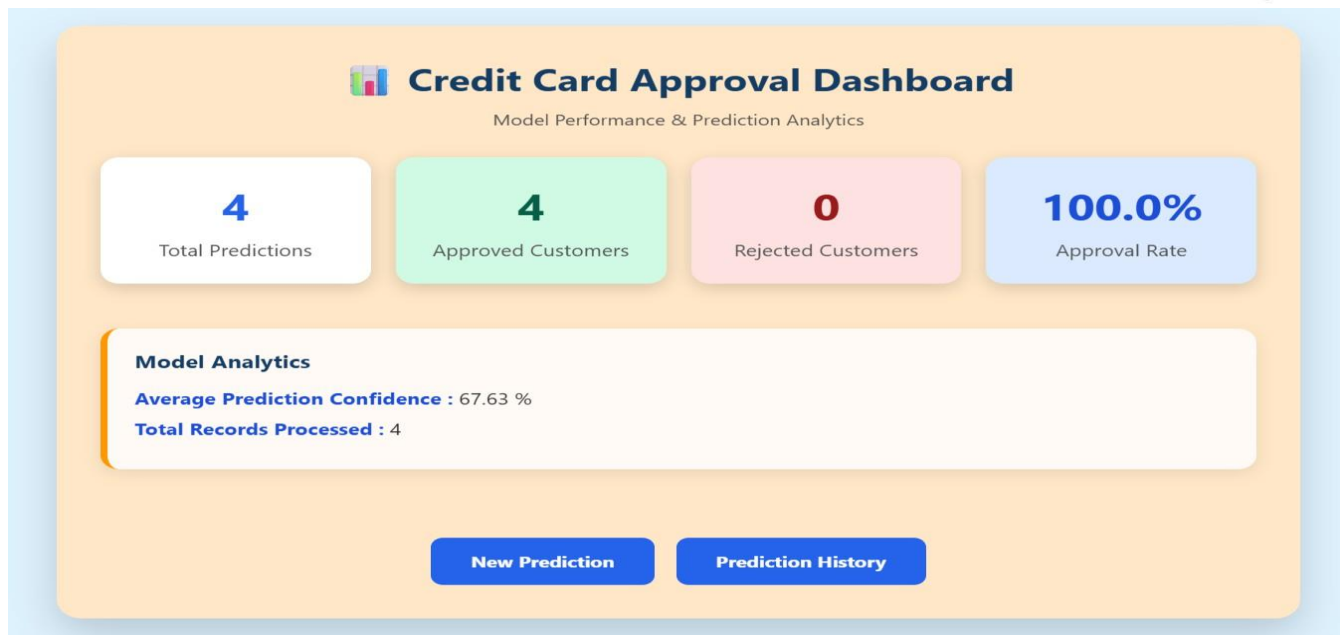
ID	Applicant Name	Prediction	Confidence	Date	Time
4	Shaik	Good Customer	69.26 %	05-07-2026	03:53 PM
3	Shaik	Good Customer	69.26 %	05-07-2026	03:51 PM
2	Shaik Jaani	Good Customer	61.65 %	04-07-2026	06:22 PM
1	Jaani	Good Customer	70.36 %	04-07-2026	12:29 PM

[← Back to Prediction](#)

Activity 5.3: Deployment Verification

After successful execution, verify that:

- The Flask server starts without errors.
- The trained Random Forest model loads successfully.
- Applicant information is accepted through the web interface.
- Predictions are generated accurately.
- Prediction records are stored in the SQLite database.
- Prediction History retrieves stored records correctly.
- Dashboard analytics display updated statistics.
- The application responds correctly across all pages.

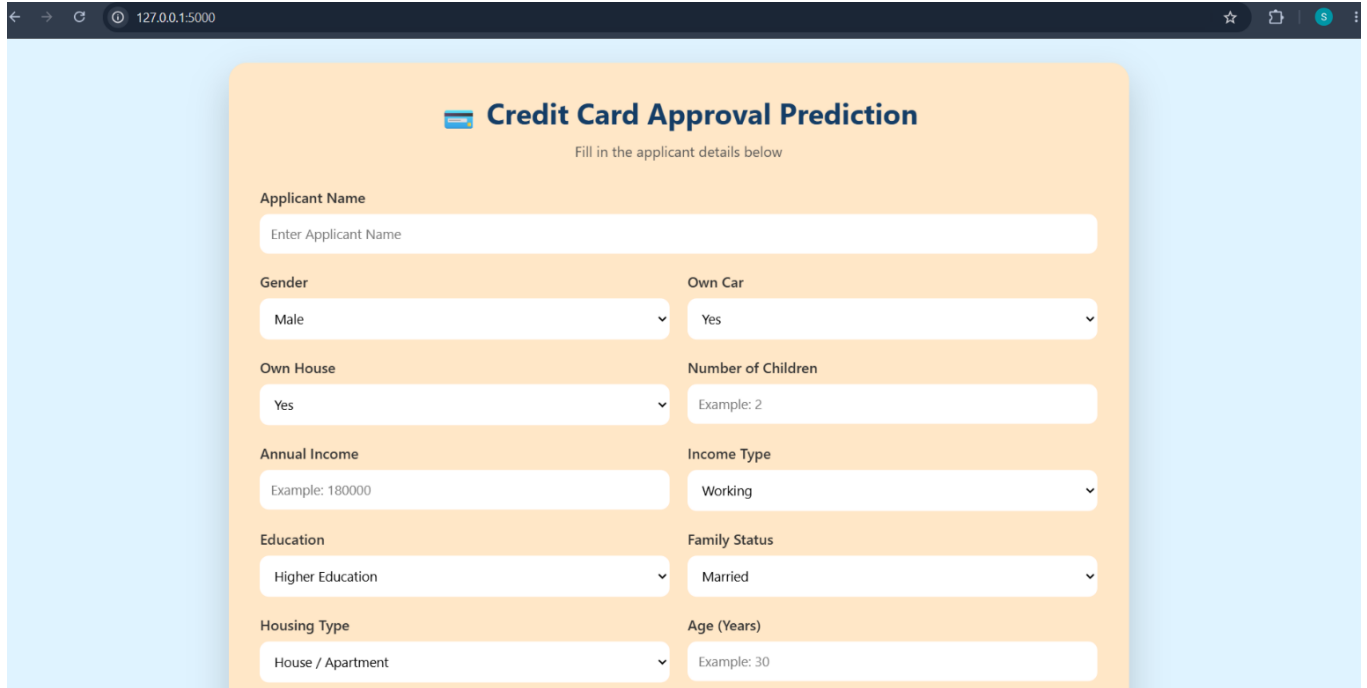


Important Notes

- The application currently runs on the **Flask Development Server** and is intended for local testing.
- The SQLite database is used to store prediction history and dashboard statistics.
- The trained **Random Forest** model (model.pkl) is loaded automatically when the application starts.
- Ensure that all required Python packages are installed before running the application.
- Future deployment can be performed using cloud platforms such as **Render**, **Railway**, **Azure**, or **AWS** for public accessibility.

Exploring the Website's Web Pages

Home Page



The screenshot shows a web browser window with the address bar displaying "127.0.0.1:5000". The main content area features a light blue background with a central orange card titled "Credit Card Approval Prediction". Below the title is a subtitle "Fill in the applicant details below". The form contains several input fields and dropdown menus:

- Applicant Name: Text input field with placeholder "Enter Applicant Name".
- Gender: Dropdown menu with "Male" selected.
- Own Car: Dropdown menu with "Yes" selected.
- Own House: Dropdown menu with "Yes" selected.
- Number of Children: Text input field with placeholder "Example: 2".
- Annual Income: Text input field with placeholder "Example: 180000".
- Income Type: Dropdown menu with "Working" selected.
- Education: Dropdown menu with "Higher Education" selected.
- Family Status: Dropdown menu with "Married" selected.
- Housing Type: Dropdown menu with "House / Apartment" selected.
- Age (Years): Text input field with placeholder "Example: 30".

Description

The **Home Page** serves as the primary interface of the **Credit Card Approval Prediction System**. It provides a clean and responsive web interface where users can enter applicant information required for credit card approval prediction. The page includes navigation options to access the **Prediction History** and **Dashboard** modules. The interface is designed using **HTML** and **CSS** to provide an intuitive and user-friendly experience.

Applicant Details Form



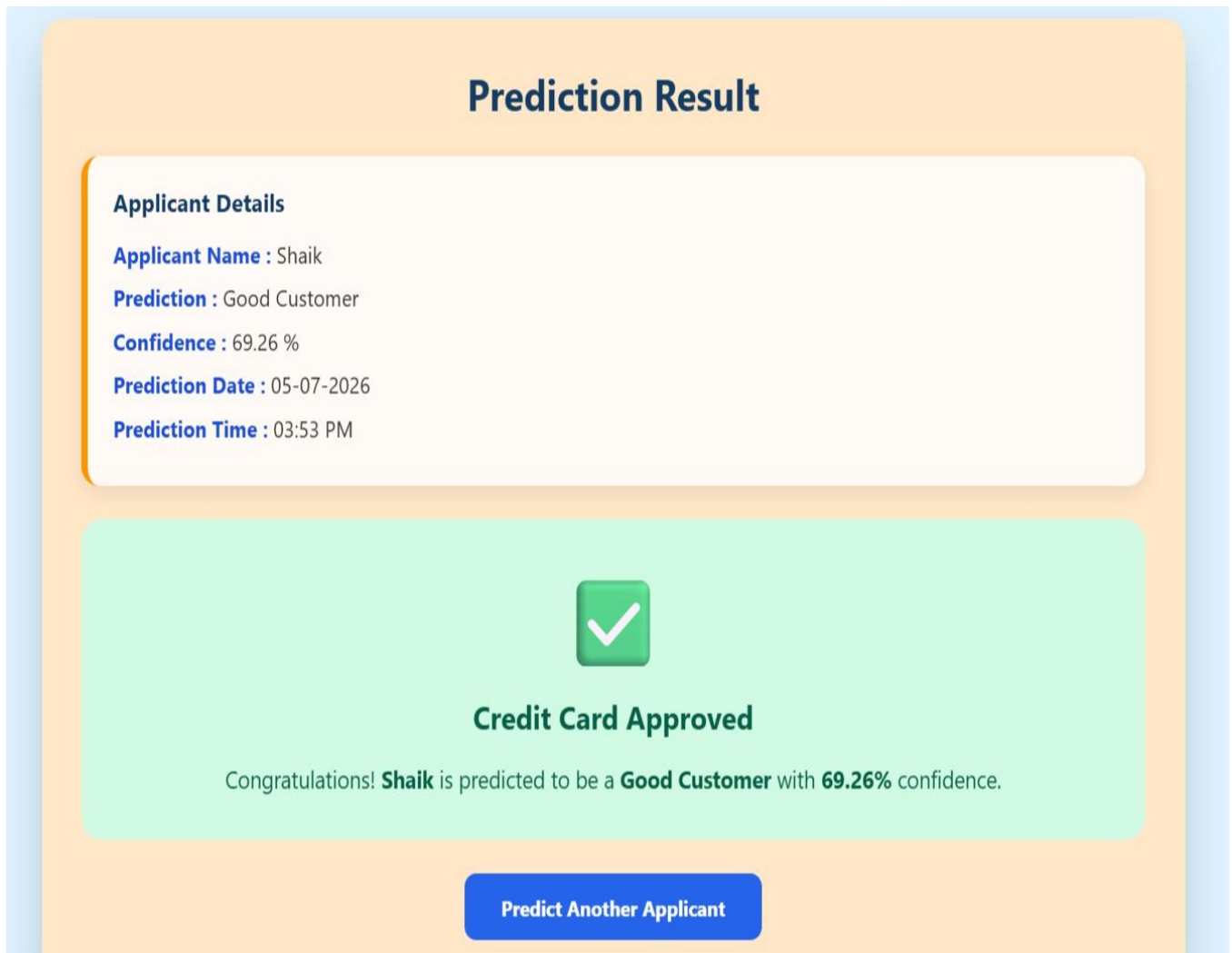
The screenshot shows the same "Credit Card Approval Prediction" form, but with sample data entered into the fields:

- Applicant Name: "Shaik"
- Gender: "Male"
- Own Car: "No"
- Own House: "No"
- Number of Children: "0"
- Annual Income: "10000"
- Income Type: "Student"
- Education: "Secondary / Secondary Special"
- Family Status: "Single / Not Married"
- Housing Type: "Rented Apartment"
- Age (Years): "21"

Description

The **Applicant Details Form** is the main input section of the application. It allows users to enter personal, financial, and employment information such as applicant name, gender, annual income, education level, family status, housing type, occupation, number of children, family members, age, and employment experience. JavaScript automatically converts **Age** and **Years Employed** into the required format before submitting the data to the Flask backend for prediction.

Prediction Result Page



The screenshot shows a web interface for displaying prediction results. At the top, a title "Prediction Result" is centered. Below it, a white box contains "Applicant Details" with the following information: Applicant Name: Shaik, Prediction: Good Customer, Confidence: 69.26 %, Prediction Date: 05-07-2026, and Prediction Time: 03:53 PM. Below this box is a large green box with a white checkmark icon, the text "Credit Card Approved", and a congratulatory message: "Congratulations! Shaik is predicted to be a Good Customer with 69.26% confidence." At the bottom of the interface is a blue button labeled "Predict Another Applicant".

Prediction Result

Applicant Details

Applicant Name : Shaik

Prediction : Good Customer

Confidence : 69.26 %

Prediction Date : 05-07-2026

Prediction Time : 03:53 PM



Credit Card Approved

Congratulations! **Shaik** is predicted to be a **Good Customer** with **69.26%** confidence.

Predict Another Applicant

Description

The **Prediction Result Page** displays the output generated by the trained **Random Forest Machine Learning model**. After processing the applicant information, the application predicts whether the applicant is a **Good Customer (Credit Card Approved)** or **Bad Customer (Credit Card Rejected)**. The page also displays the prediction confidence score, applicant name, prediction date, and prediction time, providing users with clear and detailed results.

Prediction History Page



Prediction History

All Saved Credit Card Predictions

ID	Applicant Name	Prediction	Confidence	Date	Time
4	Shaik	Good Customer	69.26 %	05-07-2026	03:53 PM
3	Shaik	Good Customer	69.26 %	05-07-2026	03:51 PM
2	Shaik Jaani	Good Customer	61.65 %	04-07-2026	06:22 PM
1	Jaani	Good Customer	70.36 %	04-07-2026	12:29 PM

← Back to Prediction

Description

The **Prediction History Page** displays all previously generated prediction records stored in the **SQLite database**. Each record includes the applicant name, prediction result, confidence score, prediction date, and prediction time. This feature enables users to review and manage historical prediction records efficiently.

Dashboard Page



Credit Card Approval Dashboard

Model Performance & Prediction Analytics

4
 Total Predictions

4
 Approved Customers

0
 Rejected Customers

100.0%
 Approval Rate

Model Analytics
 Average Prediction Confidence : 67.63 %
 Total Records Processed : 4

New Prediction

Prediction History

Description

The **Dashboard Page** provides analytical insights into the prediction records stored in the database. It displays important statistics including the **Total Predictions**, **Approved Customers**, **Rejected Customers**, **Approval Rate**, and **Average Prediction Confidence**. The dashboard helps users monitor the overall performance and usage of the prediction system through an easy-to-understand interface.

Conclusion

The **Credit Card Approval Prediction System** is a Machine Learning-based web application developed using **Python**, **Flask**, **Scikit-learn**, **Pandas**, **NumPy**, **SQLite**, **HTML**, and **CSS**. The application uses a trained **Random Forest Classifier** to predict whether a credit card applicant is likely to be approved or rejected based on personal, financial, and employment information.

The system provides an intuitive web interface for collecting applicant details and generates real-time prediction results along with a confidence score. Every prediction is automatically stored in an **SQLite database**, enabling users to review previous records through the **Prediction History** module. The application also includes a **Dashboard** that presents useful analytics such as total predictions, approved customers, rejected customers, approval rate, and average confidence score.

The project demonstrates the successful integration of **Machine Learning**, **Flask Web Development**, and **Database Management** to create an intelligent decision-support system for credit card approval prediction. Future enhancements may include deploying the application to cloud platforms, implementing advanced user authentication with role-based access control, integrating interactive data visualizations, and upgrading the database to **MySQL** or **PostgreSQL** for improved scalability and enterprise-level performance.